Semgrep php

```
syifaniads@LAPTOP-FPE63SP9:~$ semgrep scan --config p/php --include "*.php" --no-git-ignore --json --output semgrep_php.json ojs-sr

  ┌──── ○○○ ────┐
  │ Semgrep CLI │
  └─────────────┘

┌─── Loading rules from registry... ─────────────
```

```
syifaniads@LAPTOP-FPE63SP9:~$ semgrep scan --config=auto --config p/sql-injection --include "*.php" --no-git-ignore ojs-src

  ┌──── ○○○ ────┐
  │ Semgrep CLI │
  └─────────────┘

┆ Loading rules from registry...
Scanning 1347 files with 2854 Code rules:

  CODE RULES

  Language      Rules   Files          Origin      Rules
  ─────────────────────────────        ──────────────────
  php              65    1347          Pro rules    1793
  <multilang>      47    1347          Community    1061


  SUPPLY CHAIN RULES

  💎 Run `semgrep ci` to find dependency
     vulnerabilities and advanced cross-file findings.


  PROGRESS

  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:09

Warning: 1 timeout error(s) in ojs-src/lib/pkp/classes/oai/OAI.inc.php when running the following rules:
[php.lang.security.tainted-user-input-in-php-script.tainted-user-input-in-php-script]


  38 Code Findings


    ojs-src/classes/migration/upgrade/OJSv3_3_0UpgradeMigration.inc.php
    ❯❯ php.lang.security.unserialize-use.unserialize-use
          (( Blocking ))
          Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
          Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
          Details: https://sg.run/b24E

          123┆ if (@unserialize($serializedValue) === false) return;
             ⋮┆----------------------------------------
          124┆ $oldLocaleValue = unserialize($serializedValue);
             ⋮┆----------------------------------------
          148┆ if (@unserialize($serializedOldValue) === false) return;
             ⋮┆----------------------------------------
          149┆ $oldValue = unserialize($serializedOldValue);

    ojs-src/lib/pkp/classes/cache/APCCache.inc.php
    ❯❯ php.lang.security.unserialize-use.unserialize-use
          (( Blocking ))
          Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
          Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
          Details: https://sg.run/b24E

           51┆ $returner = unserialize(apc_fetch($key));

    ojs-src/lib/pkp/classes/cache/CacheManager.inc.php
    ❯❯ php.lang.security.unlink-use.unlink-use
          (( Blocking ))
          Using user input when deleting files with `unlink()` is potentially dangerous. A malicious actor
          could use this to modify or access files they have no right to.
          Details: https://sg.run/rYeR

          135┆ @unlink ($file);

    ojs-src/lib/pkp/classes/cache/FileCache.inc.php
    ❯❯ php.lang.security.unlink-use.unlink-use
          (( Blocking ))
          Using user input when deleting files with `unlink()` is potentially dangerous. A malicious actor
          could use this to modify or access files they have no right to.
          Details: https://sg.run/rYeR

           60┆ @unlink($this->filename);
```

```
syifaniads@LAPTOP-FPE63SP

  ojs-src/lib/pkp/classes/cache/XCacheCache.inc.php
  ❯❯ php.lang.security.unserialize-use.unserialize-use
        ❮❮ Blocking ❯❯
        Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
        Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
        Details: https://sg.run/b24E

         54┆ $returner = unserialize(xcache_get($key));

  ojs-src/lib/pkp/classes/cliTool/InstallTool.inc.php
 ❯❯❯ php.lang.security.backticks-use.backticks-use
        ❮❮ Blocking ❯❯
        Backticks use may lead to command injection vulnerabilities.
        Details: https://sg.run/4xj9

         97┆ @`/bin/stty -echo`;
           ⋮┆----------------------------------------
        104┆ @`/bin/stty echo`;

  ojs-src/lib/pkp/classes/context/CategoryDAO.inc.php
  ❯❯ php.lang.security.unserialize-use.unserialize-use
        ❮❮ Blocking ❯❯
        Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
        Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
        Details: https://sg.run/b24E

        139┆ $category->setImage(unserialize($row['image']));

  ojs-src/lib/pkp/classes/db/DAO.inc.php
  ❯❯ php.lang.security.unserialize-use.unserialize-use
        ❮❮ Blocking ❯❯
        Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
        Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
        Details: https://sg.run/b24E

        252┆ $value = unserialize($value);

  ojs-src/lib/pkp/classes/file/FileArchive.inc.php
 ❯❯❯ php.lang.security.exec-use.exec-use
        ❮❮ Blocking ❯❯
        Executing non-constant commands. This can lead to command injection.
        Details: https://sg.run/5Q1j

         45┆ exec(Config::getVar('cli', 'tar') . ' -c -z ' .
         46┆           '-f ' . escapeshellarg($archivePath) . ' ' .
         47┆           '-C ' . escapeshellarg($filesDir) . ' ' .
         48┆           implode(' ', array_map('escapeshellarg', array_keys($files)))
         49┆ );

  ojs-src/lib/pkp/classes/file/FileManager.inc.php
  ❯❯ php.lang.security.unlink-use.unlink-use
        ❮❮ Blocking ❯❯
        Using user input when deleting files with `unlink()` is potentially dangerous. A malicious actor
        could use this to modify or access files they have no right to.
        Details: https://sg.run/rYeR

        297┆ return unlink($filePath);
           ⋮┆----------------------------------------
        345┆ if (!unlink($file)) return false;

 ❯❯❯ php.lang.security.exec-use.exec-use
        ❮❮ Blocking ❯❯
        Executing non-constant commands. This can lead to command injection.
        Details: https://sg.run/5Q1j

        614┆ exec($gzipCmd, $output, $returnValue);

  ojs-src/lib/pkp/classes/install/Installer.inc.php
 ❯❯❯ php.lang.security.eval-use.eval-use
        ❮❮ Blocking ❯❯
        Evaluating non-constant commands. This can lead to command injection.
        Details: https://sg.run/J9AP

        383┆ return eval($condition);
           ⋮┆----------------------------------------
        431┆ return eval($condition);
```

```
syifaniads@LAPTOP-FPE63SP!

ojs-src/lib/pkp/classes/migration/upgrade/PKPv3_3_0UpgradeMigration.inc.php
❯❯ php.lang.security.unserialize-use.unserialize-use
      ❮❮ Blocking ❯❯
      Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
      Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
      Details: https://sg.run/b24E

      197┆ } elseif (@unserialize($locales) !== false ) {
        ⋮┆----------------------------------------
      198┆ $locales = unserialize($locales);
        ⋮┆----------------------------------------
      629┆ $blocks = unserialize($row->setting_value);
        ⋮┆----------------------------------------
      670┆ $blocks = unserialize($row->setting_value);

ojs-src/lib/pkp/classes/oai/PKPOAIDAO.inc.php
❯❯ php.lang.security.unserialize-use.unserialize-use
      ❮❮ Blocking ❯❯
      Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
      Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
      Details: https://sg.run/b24E

       53┆ return $row ? new OAIResumptionToken($row->token, $row->record_offset,
           unserialize($row->params), $row->expire) : null;

❯❯❯ php.lang.security.md5-loose-equality.md5-loose-equality
      ❮❮ Blocking ❯❯
      Make sure comparisons involving md5 values are strict (use `===` not `==`) to avoid type juggling
      issues
      Details: https://sg.run/Do4G

       71┆ } while($val != 0);

ojs-src/lib/pkp/classes/payment/QueuedPaymentDAO.inc.php
❯❯ php.lang.security.unserialize-use.unserialize-use
      ❮❮ Blocking ❯❯
      Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
      Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
      Details: https://sg.run/b24E

       33┆ $queuedPayment = unserialize($row->payment_data);

ojs-src/lib/pkp/classes/plugins/PluginHelper.inc.php
❯❯❯ php.lang.security.exec-use.exec-use
      ❮❮ Blocking ❯❯
      Executing non-constant commands. This can lead to command injection.
      Details: https://sg.run/5Q1j

       62┆ exec($tarBinary.' -xzf ' . escapeshellarg($filePath) . ' -C ' .
           escapeshellarg($pluginExtractDir), $output, $returnCode);

ojs-src/lib/pkp/classes/plugins/ThemePlugin.inc.php
❯❯ php.lang.security.unserialize-use.unserialize-use
      ❮❮ Blocking ❯❯
      Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
      Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
      Details: https://sg.run/b24E

      564┆ $value = $value === null ? [] : unserialize($value);

ojs-src/lib/pkp/classes/search/SearchHelperParser.inc.php
❯❯❯ php.lang.security.exec-use.exec-use
      ❮❮ Blocking ❯❯
      Executing non-constant commands. This can lead to command injection.
      Details: https://sg.run/5Q1j

       34┆ $this->fp = @popen($exec, 'r');

ojs-src/lib/pkp/classes/security/AuthSourceDAO.inc.php
❯❯ php.lang.security.unserialize-use.unserialize-use
      ❮❮ Blocking ❯❯
      Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
      Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
      Details: https://sg.run/b24E

      110┆ $auth->setSettings(unserialize($row['settings']));
```

syifaniads@LAPTOP-FPE63SP

```
 ojs-src/lib/pkp/classes/validation/ValidatorTypeDescription.inc.php
❯❯❯ php.lang.security.eval-use.eval-use
       ❮❮ Blocking ❯❯
       Evaluating non-constant commands. This can lead to command injection.
       Details: https://sg.run/J9AP

        95┆ $validator = eval($validatorConstructorCode);

 ojs-src/lib/pkp/classes/xslt/XSLTransformer.inc.php
❯❯❯ php.lang.security.exec-use.exec-use
       ❮❮ Blocking ❯❯
       Executing non-constant commands. This can lead to command injection.
       Details: https://sg.run/5Q1j

       204┆ exec($xsltCommand . ' 2>&1', $contents, $status);

 ojs-src/lib/pkp/controllers/grid/plugins/PluginGalleryGridHandler.inc.php
❯❯ php.lang.security.unlink-use.unlink-use
       ❮❮ Blocking ❯❯
       Using user input when deleting files with `unlink()` is potentially dangerous. A malicious actor
       could use this to modify or access files they have no right to.
       Details: https://sg.run/rYeR

       272┆ unlink($destPath);
         ⋮┆----------------------------------------
       285┆ unlink($destPath);

 ojs-src/lib/pkp/pages/admin/AdminHandler.inc.php
❯❯❯ php.lang.security.phpinfo-use.phpinfo-use
       ❮❮ Blocking ❯❯
       The 'phpinfo' function may reveal sensitive information about your environment.
       Details: https://sg.run/W82E

       375┆ phpinfo();

 ojs-src/lib/pkp/pages/install/InstallHandler.inc.php
❯❯❯ php.lang.security.search-active-debug.search-active-debug
       ❮❮ Blocking ❯❯
       Debug logging is explicitly enabled. This can potentially disclose sensitive information and should
       never be active on production systems.
       Details: https://sg.run/PKeW

        33┆ @ini_set('display_errors', true);

 ojs-src/lib/pkp/tools/copyAccessLogFileTool.php
❯❯❯ php.lang.security.exec-use.exec-use
       ❮❮ Blocking ❯❯
       Executing non-constant commands. This can lead to command injection.
       Details: https://sg.run/5Q1j

       204┆ exec($egrepPath . " -i '" . $this->_contextPaths . "' " . escapeshellarg($tmpFilePath) . "
            > " . escapeshellarg($destinationPath), $output, $returnValue);

 ojs-src/plugins/generic/usageStats/UsageStatsPlugin.inc.php
❯❯❯ php.lang.security.mcrypt-use.mcrypt-use
       ❮❮ Blocking ❯❯
       Mcrypt functionality has been deprecated and/or removed in recent PHP versions. Consider using
       Sodium or OpenSSL.
       Details: https://sg.run/BkZR

       698┆ $newSalt = bin2hex(mcrypt_create_iv(16, MCRYPT_DEV_URANDOM|MCRYPT_RAND));

 ojs-src/plugins/importexport/datacite/DataciteExportPlugin.inc.php
❯❯❯ php.lang.security.exec-use.exec-use
       ❮❮ Blocking ❯❯
       Executing non-constant commands. This can lead to command injection.
       Details: https://sg.run/5Q1j

       384┆ exec($tarCommand);


Scan Summary

✅ Scan completed successfully.
```

```
Scan Summary

✅ Scan completed successfully.
 • Findings: 38 (38 blocking)
 • Rules run: 112
 • Targets scanned: 1347
 • Parsed lines: ~100.0%
 • Scan skipped:
   ◦ Not matching --include patterns: 10213
   ◦ Files matching .semgrepignore patterns: 138
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 112 rules on 1347 files: 38 findings.

📢 Too many findings? Try Semgrep Pro for more powerful queries and less noise.
   See https://sg.run/false-positives.
syifaniads@LAPTOP-FPE63SP9:~$
```

Ditemukan:

1. SQL INJECTION (TIDAK ADA)
2. Command injection

```
exec($gzipCmd, $output, $returnValue);
popen($exec, 'r');
`/bin/stty -echo`
```

3. CODE INJECTION (EVAL)

```
eval($condition);
```

4. DESERIALIZATION (RCE)

```
unserialize($value);
```

5. FILE-BASED (unlink)

```
unlink($filePath);
```

Semgrep owasp10

```
syifaniads@LAPTOP-FPE63SP9:~$ semgrep scan --config p/owasp-top-ten --config p/security-audit --include "*.php" --no-git-ignore --json --output semgrep_owasp.json
ojs-src

┌─── ooo ───┐
│ Semgrep CLI │
└─────────────┘

ℹ Loading rules from registry...                                                                                    Scanning 1347 files with 2413
Code rules:

  CODE RULES

  Language      Rules   Files          Origin      Rules
  ─────────────────────────────        ────────────────────
  php              49    1347          Pro rules    1739
  <multilang>       8    1347          Community     674


  SUPPLY CHAIN RULES

  💎 Run `semgrep ci` to find dependency
     vulnerabilities and advanced cross-file findings.


  PROGRESS

  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:10


┌─────────────┐
│ Scan Summary │
└─────────────┘
✅ Scan completed successfully.
 • Findings: 2 (2 blocking)
 • Rules run: 57
 • Targets scanned: 1347
 • Parsed lines: ~100.0%
 • Scan skipped:
   ◦ Not matching --include patterns: 10213
   ◦ Files matching .semgrepignore patterns: 138
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 57 rules on 1347 files: 2 findings.
syifaniads@LAPTOP-FPE63SP9:~$
```

Hasil temuan 10 semgrep auto

No	Rule	File	Line	Severity	Kategori
1	unserialize-use	OJSv3_3_0UpgradeMigration.inc.php	123	WARNING	Deserialization
2	unserialize-use	OJSv3_3_0UpgradeMigration.inc.php	124	WARNING	Deserialization
3	unserialize-use	OJSv3_3_0UpgradeMigration.inc.php	148	WARNING	Deserialization
4	unserialize-use	OJSv3_3_0UpgradeMigration.inc.php	149	WARNING	Deserialization
5	unserialize-use	APCCache.inc.php	51	WARNING	Deserialization
6	unlink-use	CacheManager.inc.php	135	WARNING	File Manipulation
7	unlink-use	FileCache.inc.php	60	WARNING	File Manipulation

8	unserialize-use	XCacheCache.inc.php	54	WARNING	Deserialization
9	backticks-use	InstallTool.inc.php	97	ERROR	Command Injection
10	backticks-use	InstallTool.inc.php	104	ERROR	Command Injection

Custom rule — Deteksi `eval()` berbahaya:

```
syifaniads@LAPTOP-FPE63SP9:~$ semgrep scan --config custom_rules.yaml --include "*.php" --no-git-ignore --json --output semgrep_custo
m.json ojs-src

┌─────────────┐
│ Semgrep CLI │
└─────────────┘

Scanning 1347 files with 3 Code rules:

  CODE RULES
  Scanning 1347 files with 3 php rules.

  SUPPLY CHAIN RULES

  💎 Run `semgrep ci` to find dependency
     vulnerabilities and advanced cross-file findings.

  PROGRESS

  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:07


┌─────────────┐
│ Scan Summary │
└─────────────┘

  ✅ Scan completed successfully.
  • Findings: 124 (124 blocking)
  • Rules run: 3
  • Targets scanned: 1347
  • Parsed lines: ~100.0%
  • Scan skipped:
    ◦ Not matching --include patterns: 10213
    ◦ Files matching .semgrepignore patterns: 138
  • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 3 rules on 1347 files: 124 findings.

📢 Too many findings? Try Semgrep Pro for more powerful queries and less noise.
   See https://sg.run/false-positives.
```

```
syifaniads@LAPTOP-FPE63SP9:~$ python3 -c "
import json

with open('semgrep_custom.json') as f:
    data = json.load(f)

results = data.get('results', [])

filtered = [r for r in results if r['extra']['severity'] == 'ERROR']

for r in filtered[:10]:
    print(f\"{r['check_id']} | {r['path']}:{r['start']['line']}\")
"
php-eval-injection | ojs-src/lib/pkp/classes/install/Installer.inc.php:383
php-eval-injection | ojs-src/lib/pkp/classes/install/Installer.inc.php:431
php-eval-injection | ojs-src/lib/pkp/classes/validation/ValidatorTypeDescription.inc.php:95
syifaniads@LAPTOP-FPE63SP9:~$ python3 -c "
import json

with open('semgrep_custom.json') as f:
    data = json.load(f)

results = data.get('results', [])

filtered = [
    r for r in results
    if r['extra']['severity'] in ['ERROR', 'WARNING']
]

for r in filtered[:15]:
    print(f\"{r['extra']['severity']} | {r['check_id']} | {r['path']}:{r['start']['line']}\")
"
WARNING | php-unserialize-user-input | ojs-src/classes/migration/upgrade/OJSv3_3_0UpgradeMigration.inc.php:123
WARNING | php-unserialize-user-input | ojs-src/classes/migration/upgrade/OJSv3_3_0UpgradeMigration.inc.php:124
WARNING | php-unserialize-user-input | ojs-src/classes/migration/upgrade/OJSv3_3_0UpgradeMigration.inc.php:148
WARNING | php-unserialize-user-input | ojs-src/classes/migration/upgrade/OJSv3_3_0UpgradeMigration.inc.php:149
WARNING | php-file-include-variable | ojs-src/index.php:65
WARNING | php-unserialize-user-input | ojs-src/lib/pkp/classes/cache/APCCache.inc.php:51
WARNING | php-file-include-variable | ojs-src/lib/pkp/classes/cache/FileCache.inc.php:46
WARNING | php-unserialize-user-input | ojs-src/lib/pkp/classes/cache/XCacheCache.inc.php:54
WARNING | php-file-include-variable | ojs-src/lib/pkp/classes/cliTool/CliTool.inc.php:32
WARNING | php-unserialize-user-input | ojs-src/lib/pkp/classes/context/CategoryDAO.inc.php:139
WARNING | php-file-include-variable | ojs-src/lib/pkp/classes/core/APIRouter.inc.php:93
WARNING | php-file-include-variable | ojs-src/lib/pkp/classes/core/APIRouter.inc.php:113
WARNING | php-file-include-variable | ojs-src/lib/pkp/classes/core/PKPApplication.inc.php:135
WARNING | php-file-include-variable | ojs-src/lib/pkp/classes/core/PKPPageRouter.inc.php:203
WARNING | php-file-include-variable | ojs-src/lib/pkp/classes/core/PKPPageRouter.inc.php:205
```

phpcs-security-audit

```
syifaniads@LAPTOP-FPE63SP9:~$ phpcs --standard=Security --extensions=php --ignore=*/vendor/*,*/node_modules/* ojs-src > phpcs_securit
y_output.txt 2>&1
syifaniads@LAPTOP-FPE63SP9:~$ head phpcs_security_output.txt

FILE: /home/syifaniads/ojs-src/pages/article/ArticleHandler.inc.php
----------------------------------------------------------------------
FOUND 0 ERRORS AND 2 WARNINGS AFFECTING 2 LINES
----------------------------------------------------------------------
 228 | WARNING | Function array_map() that supports callback detected
 232 | WARNING | Function array_map() that supports callback detected
----------------------------------------------------------------------

syifaniads@LAPTOP-FPE63SP9:~$
```